

Lab 10 - Motion, Tracking and Temporal Continuity Measurement

Course: Generative AI for Image and Video Creation (TGS-2020505925) | Version v11.0 | 6 September 2026 Topic 04: Generative AI Video Creation, Animation and Editing Outcome: ELO5 | In-class time: 60 minutes of scheduled practical time

Competency	Statement
Ability A6	Design and apply video analytics algorithms for high-level video analytics tasks
Knowledge K10	Activity tracking, generative models, scene understanding and event discovery

Scenario

A delivered clip 'feels jittery' and 'the product seems to drift'. Turn both complaints into numbers. Isolate the moving foreground with background subtraction, track the subject with mean shift, CAMShift and a learning tracker, measure the dense motion field with optical flow, and report centroid jitter, scale drift and inter-frame structural similarity.

What you will produce

out/tracking.csv with per-frame centroid and box size for three trackers, out/continuity-report.json with jitter, drift and inter-frame SSIM statistics, and annotated overlay videos for each tracker.

Tools: Python 3 | opencv-python | NumPy | no network, no paid service

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: python3, opencv-python, numpy.

API currency. Everything taught here runs on the base opencv-python wheel. Two things that appear in older tutorials are *not* in that wheel and are deliberately avoided: cv2.TrackerCSRT_create / cv2.TrackerKCF_create and the cv2.quality SSIM module, both of which live in opencv-contrib-python. Where SSIM is needed the course ships its own NumPy implementation.

Workflow

1. Subtract the background to isolate motion
2. Track with mean shift and CAMShift
3. Track with the built-in MIL tracker

4. Compute dense optical flow statistics
5. Report jitter, scale drift and inter-frame SSIM

Files in this folder

Path	What it is
reference/nq-motion-clip.mp4	Synthetic 8-second 640x480 25 fps clip: a subject block translates across a static shelf background while growing in scale, with a deliberate two-frame position discontinuity at t=4.0 s (deterministically rendered with OpenCV - SIMULATED, not generative-model output)
reference/nq-motion-truth.json	The exact per-frame subject box used to render the clip, so every tracking error is measurable rather than estimated
data/tracking-config.json	The initial track window, the tracker list, and the continuity thresholds
data/continuity-thresholds.json	Pass/fail limits for centroid jitter, scale drift and inter-frame SSIM
motion.py	Runnable script - see the steps below
ssim.py	Shared helper, copied into this folder so it is self-contained
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom asset rendered deterministically with OpenCV, an AI-generated reference copied unchanged with a PROVENANCE.md beside it, or a real prerecorded sequence with its source and SHA-256 recorded. Nothing in this folder may be relabelled: a rendered animation is never presented as model output, and a simulated stand-in is never presented as a photograph.

Steps

1. Open the clip and confirm its properties before anything else: 640x480, 25 fps, 200 decodable frames, 8.00 s. Lab 02 taught you not to trust the container.

```
python3 -c "import cv2;c=cv2.VideoCapture('reference/nq-motion-clip.mp4');print(int(c.get(3)),int(c.get(4)),c.get(5),int(c.get(7)))"
```

2. Run background subtraction with `cv2.createBackgroundSubtractorMOG2()`. MOG2 models each pixel as a mixture of Gaussians over time and labels a pixel foreground when it no longer fits its learned model. Report the foreground pixel count per frame.

```
sub = cv2.createBackgroundSubtractorMOG2(history=200, varThreshold=16,
                                         detectShadows=True)
fg = sub.apply(frame)
```

3. Note that `detectShadows=True` labels shadow pixels 127 rather than 255. Threshold at 200 to get a hard foreground mask, or you will count shadows as subject.

```
_, hard = cv2.threshold(fg, 200, 255, cv2.THRESH_BINARY)
```

4. Repeat with `cv2.createBackgroundSubtractorKNN()` and compare the foreground counts over the first 50 frames. Record which stabilises faster - the model needs history before it is reliable, which is why the first frames of any BS pipeline are untrustworthy.

5. Explain background subtraction in one paragraph for your report: it builds a per-pixel model of the static scene from recent history and flags pixels that deviate from it, producing a binary foreground mask. It requires a static camera; a pan invalidates the model.
6. Set up mean shift. Take the initial window from data/tracking-config.json, compute the hue histogram of that region, and back-project it onto each frame. Mean shift then climbs the density of the back-projection to find the window's new position.

```
roi_hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(roi_hsv, (0, 60, 32), (180, 255, 255))
hist = cv2.calcHist([roi_hsv], [0], mask, [180], [0, 180])
cv2.normalize(hist, hist, 0, 255, cv2.NORM_MINMAX)
```

7. Run `cv2.meanShift(back_projection, window, term_crit)` frame by frame and record the window each time. Note the defining limitation: the window size never changes, so as the subject grows the window stops containing it.

```
term = (cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 1)
ret, window = cv2.meanShift(bp, window, term)
```

8. Run `cv2.CamShift` with the identical setup. CAMShift re-computes the window size from the zeroth moment of the back-projection each iteration and also returns an orientation, so it adapts to a subject that changes scale.

```
rot_rect, window = cv2.CamShift(bp, window, term)
```

9. Quantify the difference. The clip's subject grows by a known factor between the first and last frame - read it from reference/nq-motion-truth.json. Report each tracker's final box area as a ratio of its initial area, and compare against the truth ratio. This is the concrete evidence for when to choose CAMShift over mean shift.

10. Run the third tracker, `cv2.TrackerMIL_create()`. Note the API-currency point: `cv2.TrackerCSRT_create` and `cv2.TrackerKCF_create` are NOT in the base opencv-python wheel - they require opencv-contrib-python. TrackerMIL is built in, which is why this lab uses it.

```
tracker = cv2.TrackerMIL_create()
tracker.init(first_frame, tuple(init_window))
ok, box = tracker.update(frame)
```

11. Run all three trackers over the clip and write out/tracking.csv with, per frame and per tracker: centre x, centre y, width, height, and the IoU against the ground-truth box.

```
python3 motion.py --video reference/nq-motion-clip.mp4 --truth reference/nq-motion-truth.
json --config data/tracking-config.json --out out
```

12. Compute dense optical flow with `cv2.calcOpticalFlowFarneback` between consecutive grayscale frames. The result is an $H \times W \times 2$ array of per-pixel (dx, dy) displacement. Convert to magnitude and angle with `cv2.cartToPolar` and report the mean magnitude per frame.

```
flow = cv2.calcOpticalFlowFarneback(prev_gray, gray, None,
                                    0.5, 3, 15, 3, 5, 1.2, 0)
mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])
```

13. Find the discontinuity. The clip contains a deliberate two-frame position jump at $t = 4.0$ s (frame 100). Inspect whether it appears as a spike in mean flow magnitude and as a spike in centroid step size. These are candidate flags, not guaranteed detectors; inspect the clip and tracker IoU to explain any missed jump.

14. Compute the three continuity metrics. CENTROID JITTER: the standard deviation of the frame-to-frame centroid step, excluding only explicitly declared transition indices in `config.excluded_transition_frames`. SCALE DRIFT: the absolute difference between the tracker's area ratio and the ground truth's. INTER-FRAME SSIM: the mean SSIM

between consecutive frames, which falls sharply at a cut or flicker in some clips; static backgrounds can dominate this score.

15. Reuse the SSIM implementation from Lab 05 rather than writing a second one - a shared copy of `ssim.py` is included in this lab folder for self-containment. Confirm it is byte-identical to the Lab 05 copy so the two labs cannot disagree.

```
python3 -c "import hashlib;print(hashlib.sha256(open('ssim.py','rb').read()).hexdigest()[:16])"
```

16. Read `data/continuity-thresholds.json` and mark each metric PASS or FAIL. Report the jitter/drift verdict per tracker and SSIM once for the source clip. Tracking errors and real clip defects can both affect results; use truth IoU and visual review to distinguish them.

17. Write the interpretation paragraph. Connect it back to generative video: temporal coherence is the property that distinguishes a video model from a per-frame image model, and it is why architectures extend the 2-D U-Net to 3-D or treat video as a sequence of spatiotemporal patches. Your jitter and inter-frame SSIM numbers are the simple diagnostic proxies. FVD instead compares distributions of learned video features; these scores neither compute FVD nor certify semantic coherence.

18. Save `out/tracking.csv`, `out/continuity-report.json` and the three overlay videos, then complete the evidence checklist.

Verify

`out/tracking.csv` has a row per frame for all three trackers with an IoU against ground truth, `out/continuity-report.json` reports centroid jitter, scale drift and clip-level mean inter-frame SSIM, plus per-tracker jitter/drift PASS/FAIL, and you have located the frame-100 discontinuity in both the optical-flow magnitude and the centroid step series.

Expected outputs

- `out/tracking.csv` - 200 frames x 3 trackers with `cx`, `cy`, `w`, `h` and `iou_vs_truth`.
- Mean shift final/initial box area ratio ~ 1.0 (the window never resizes).
- CAMShift area ratio tracking the ground-truth growth more closely if tracking remains valid; verify rather than assume.
- Inspect flow/centroid candidates around frame 100; large displacements may violate flow assumptions or be diluted by static background.
- `out/continuity-report.json` - jitter/scale drift per tracker and clip-level inter-frame SSIM with PASS/FAIL against the thresholds.
- `out/track_meanshift.mp4`, `out/track_camshift.mp4`, `out/track_mil.mp4` overlays.

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- ☐ `out/continuity-report.json` in full.
- ☐ The mean shift and CAMShift area ratios next to the ground-truth ratio.
- ☐ The frame index of the detected discontinuity, from both the flow and centroid series.
- ☐ Mean IoU against ground truth for each of the three trackers.
- ☐ One paragraph explaining background subtraction and its static-camera requirement.
- ☐ The API-currency note: which tracker constructors are absent from the base wheel.
- ☐ The interpretation paragraph linking your metrics to temporal coherence in video models.

Troubleshooting

Symptom	What to do
cv2.TrackerCSRT_create raises AttributeError	Expected on the base opencv-python wheel - CSRT and KCF live in opencv-contrib-python. Use cv2.TrackerMIL_create(), which is built in. Do not add a contrib dependency for this lab.
Mean shift locks onto the background	The initial window includes too much background, so the hue histogram is dominated by it. Shrink the initial window to the subject, and keep the cv2.inRange(..., (0,60,32), (180,255,255)) saturation/value floor that excludes washed-out pixels.
CAMShift collapses the window to a few pixels	The back-projection has almost no support - usually because the subject left the frame or changed hue. Re-seed from the ground-truth box and report the frame where it collapsed as a finding.
Optical flow magnitude is enormous everywhere	You passed colour frames. calcOpticalFlowFarneback requires single-channel 8-bit input; convert both frames with cv2.COLOR_BGR2GRAY first.
Inter-frame SSIM is 1.0 for every pair	You are comparing a frame with itself - check the previous-frame variable is actually being updated at the end of the loop.
The discontinuity does not show in the centroid series	Your tracker lost the subject before frame 100, so there is no step to measure. Check the IoU column: once it hits zero the series after it is meaningless, and you must say so rather than reporting the numbers.
Overlay videos are empty	Same cause as Lab 09: a frame written to the VideoWriter with a different shape from the declared size. Resize every frame before writing.

References used in this lab

- [S10] Tschochohei and Schenker (2026), as above - *Ch.7 pp.106-109*.

Spatial versus temporal coherence; early video-GAN failure modes (temporal artifacts/flicker, inconsistent motion, mode collapse) and VGAN/MoCoGAN/TGAN; the 2-D to 3-D U-Net extension for spatiotemporal features; Diffusion Transformers treating video as a sequence of spatiotemporal patches; the 'consistency bottleneck' argument for diffusion over GANs; platform comparison table (core technology, max duration and resolution, primary use case).

- [S17] Mewada et al. (2026), as above - *Ch.7 pp.94-98 (Dubey, Pinheiro, Singh, Ansari and Kumar, 'Advanced Foundations and Future Trends in Generative AI for Visual Media')*.

Table 7.2 scaling strategies and their targets (trajectory distillation -> fewer sampling steps at similar FID/FVD; quantisation 8/4-bit -> reduced latency and memory; low-rank adapters -> small trainable-parameter fraction; latent diffusion -> lower bandwidth cost at high resolution; retrieval conditioning -> higher faithfulness without more parameters). Evaluation protocol: FID as a 2-Wasserstein approximation in feature space (encoder-dependent, sample-size sensitive), KID as an unbiased MMD², LPIPS as a learned perceptual distance, precision/recall for mode coverage, CLIP-based text-image faithfulness, masked-region consistency and identity preservation for edits, FVD plus optical-flow agreement for video, and watermark detectability under common edits (Table 7.3 axes and pitfalls).

- [S23] OpenCV 4.13 Python package, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92)*.

API-currency verification for every command taught: cv2.data.harcascades ships the frontal-face and eye cascades so no weights download is required; cv2.HOGDescriptor_getDefaultPeopleDetector() is built in; SIFT_create, ORB_create, FastFeatureDetector_create, cornerHarris, goodFeaturesToTrack, matchTemplate, createBackgroundSubtractorMOG2/KNN, meanShift, CamShift, calcOpticalFlowFarneback and PSNR are all present. IMPORTANT CORRECTION carried into the labs: cv2.TrackerCSRT_create and cv2.TrackerKCF_create are NOT in the base opencv-python wheel (they require opencv-contrib-python), and the cv2.quality SSIM module is also contrib-only - so the labs use cv2.TrackerMIL_create and a NumPy SSIM implementation to stay dependency-light and runnable offline.

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v11.0.